zeroriscy_controller

OVERVIEW

The Controller is one of the most important modules in the Zero-Riscy processor. It acts as the central dispatcher of the CPU, managing the overall control flow of instruction execution across all pipeline stages.
Unlike the ALU or Register File, the controller does not process data directly. Instead, it manages the processor's internal state machine and ensures that all pipeline stages work in proper coordination.
Its primary responsibility is to control instruction flow, synchronize execution stages, handle interrupts and exceptions, manage stalls, and resolve hazards that may affect processor performance.

In simple terms: Controller = Traffic Manager of the CPU

It decides:

- When to fetch instructions
- When to stop or stall the pipeline
- When to jump to a new PC address
- When to handle interrupts or exceptions
- When to enter debug mode
- When forwarding is required to avoid hazards

CORE RESPONSIBILITIES

The controller performs several major functions inside the processor:

- Synchronizing all pipeline stages
- Managing fetch, decode, and execution flow
- Handling exceptions and interrupts
- Supporting debug halt and resume
- Controlling stalls and hazard resolution
- Managing forwarding paths
- Generating performance monitoring signals

This makes the controller the decision-making center of the processor.

INSTRUCTION FETCH (IF) CONTROL

Goal:
The main goal of the Instruction Fetch stage is to continuously supply valid instructions from memory to the CPU without delay. The controller works closely with the prefetcher to ensure smooth instruction delivery.

Important Input Signal:

- instr_valid_i -> Indicates that the prefetcher has a valid instruction ready. This acts like an 'I am ready' signal from the fetch unit.

Important Output Signals:

- instr_req_o -> Master ON switch for instruction memory requests. When active, the controller allows the prefetcher to fetch instructions from memory. If the processor enters reset or sleep mode, this signal is disabled to save power.

- pc_set_o -> Program Counter redirect signal. Used when the PC must jump to a new address instead of continuing with normal sequential execution. Examples: Branch taken, Jump instruction, Exception handling, Interrupt handling.

- pc_mux_o -> Selects the source of the next PC value. It chooses between: Normal PC + 4, Jump target, Exception handler address, Boot address.

- halt_if_o -> Emergency brake for fetch stage. Stops all instruction fetching when required. Used during: Sleep mode, Debug halt, Pipeline stall conditions.

INSTRUCTION DECODE (ID) CONTROL

Goal:

The purpose of the decode stage is to translate raw instruction bits into meaningful CPU operations. The controller supervises this process and ensures safe execution.

Important Input Signals:

- Instruction Type Signals (jump_set_i, ecall_insn_i, illegal_insn_i) -> These help identify special conditions like jump instructions, system calls, and illegal instructions.

- id_ready_i -> Green Light signal from decode stage. It tells the controller: 'I have finished processing this instruction.' Only then can the next instruction move forward.

Important Output Signals:

- is_decoding_o -> Permission signal to begin decoding. This formally starts the decode operation.

- halt_id_o -> Freeze current instruction. Used when: Multi-cycle operations are running (like division), Waiting for interrupt handling, Hazard resolution is required.

- deassert_we_o -> Safety protection signal. Disables register writes when errors such as illegal instructions are detected. This prevents incorrect data from being written into the register file.

INTERRUPT AND DEBUG CONTROL

Interrupt Handling:

The controller constantly monitors external hardware requests.

- irq_req_ctrl_i -> External interrupt request input. This means: 'Some external hardware needs CPU attention.' Examples: Timer interrupt, External peripheral request.

- irq_ack_o -> Interrupt acknowledge output. This confirms that the current instruction flow is being stopped and interrupt handling will begin. The controller then redirects execution to the interrupt handler.

Debug Handling:

The controller also supports external debugging through tools like GDB.

- dbg_req_i -> External debugger pause request. This means: 'The developer wants to pause the processor.'

- dbg_ack_o -> Debug acknowledgment output. This confirms that: Pipeline is cleared, CPU is safely paused, Debug mode is active. This is critical for testing and verification.

LOAD STORE UNIT (LSU) INTERACTION

Goal:
The controller helps manage correct data movement between processor and memory.

Major Challenge: Misaligned Data
Sometimes memory addresses are not aligned to 4-byte boundaries. This creates problems during load/store operations.

- data_misaligned_i -> LSU detects misaligned address. This tells the controller that normal execution cannot continue safely.
- operand_a_fw_mux_sel_o -> Forwarding control signal. The controller activates a shortcut path to reroute corrected data directly into execution.

This forwarding mechanism prevents unnecessary stalls, avoids incorrect register reads, and ensures correct execution.

PREFETCHING MECHANISM

The prefetcher works like a scout that stays ahead of the CPU. It fetches instructions early and stores them in a small queue so the processor does not need to wait for memory access.

Signal Relationship:

- instr_req_o (Controller to Prefetcher) -> Enables instruction fetching.
- instr_valid_i (Prefetcher to Controller) -> Indicates that a valid instruction is ready.
- pc_set_o + pc_mux_o (Controller to Prefetcher) -> Used for redirecting instruction fetch.

Normally, PC = PC + 4. If a branch or jump occurs, pc_set_o = 1. This tells the prefetcher: 'Discard current fetch and jump to the new address.' This improves instruction delivery speed and reduces waiting time.

STALL SIGNALS (PIPELINE BRAKES)

What is a Stall?
A stall occurs when the CPU must pause one stage because the next stage is not ready. This prevents incorrect execution.

- halt_if_o -> Stop fetching new instructions. Used during: Sleep mode, Debug halt, Exception handling.
- halt_id_o -> Freeze current decode instruction. Used for: Multi-cycle operations, Interrupt waiting, Hazard resolution.
- id_ready_i -> Decode completion signal. This tells the controller: 'You can safely move the pipeline forward.'

Stalls ensure safe and correct instruction execution.

FORWARDING SIGNALS (SHORTCUTS)

What is Forwarding?
Forwarding is used to avoid stalls caused by data dependency hazards. Instead of waiting for data to be written back to the register file, the result is directly forwarded to the next instruction. This improves pipeline efficiency.

- operand_a_fw_mux_sel_o -> Selects the forwarding source.

In this design, it is linked to data_misaligned_i. Meaning: If misaligned data is detected, the controller switches the multiplexer and routes corrected LSU data directly into execution. This avoids using incorrect register values and reduces stall cycles.

PERFORMANCE AND STATUS SIGNALS

These signals help developers and the system monitor processor activity.

- perf_jump_o -> Pulse generated when a jump occurs
- perf_tbranch_o -> Pulse generated when a branch is successfully taken

These are connected to performance counters for measuring: IPC (Instructions Per Cycle), Branch efficiency, and Jump frequency.

- ctrl_busy_o -> Controller activity status signal. Acts like a system status light. HIGH = Controller is active, LOW = CPU is idle or in sleep mode. Example: During WFI (Wait For Interrupt), this signal becomes LOW.

SUMMARY TABLE

Block | Controller Listens For -> Controller Commands

- Fetch Unit | Is instruction valid? -> Fetch next, Jump, or Halt
- Decoder | Is it a jump? Is it illegal? -> Start decoding, Stop, Clear errors
- Interrupts | Is there an emergency? -> Acknowledge and switch to handler
- Debug | Does user want to stop? -> Confirm pause and report status

FINAL SUMMARY

The zeroriscy_controller is the brain of pipeline control inside the Zero-Riscy processor.
It:

- Controls fetch, decode, and execution flow
- Handles interrupts and exceptions
- Manages stalls and forwarding
- Supports debugging operations
- Coordinates prefetching and PC redirection
- Generates performance monitoring signals

Although it does not perform arithmetic operations, it is responsible for ensuring that every part of the processor works in the correct sequence, safely and efficiently. Without the controller, the processor cannot function as an organized and reliable system.